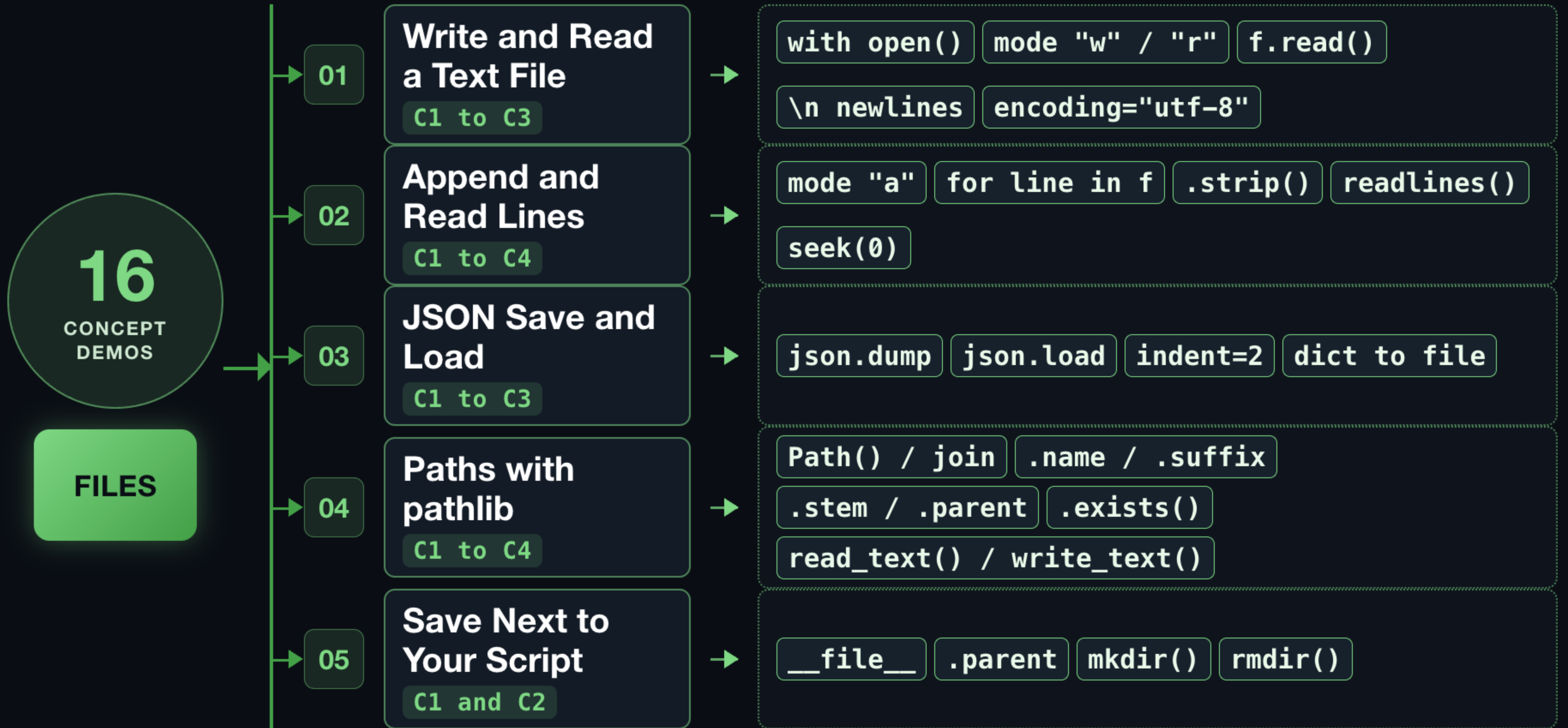


Section 10: Files, JSON, and Paths **Concept Map**

The full path through this section - we open here before each concept, then dive in.



Files: Keywords Covered

Section 10: Files, JSON and Paths - the keywords in bright green are the new ones we cover here.

Values TrueFalseNone	Classes class	Context Managers with
Logic & Membership andornotinis	Error Handling tryexceptfinallyraise assert	Cleanup & Flow passdel
Conditionals ifelifelse	Modules & Imports importfromas	Generators & Async yieldasyncawait
Loops forwhilebreakcontinue	Scope globalnonlocal	Soft Keywords matchcase_ type
Functions defreturnlambda		

keyword

New in this section

keyword

Covered earlier

keyword

Not covered yet

Files: the **mode** decides everything

One letter says what you are about to do to the file

You pass it as the second argument to **open**. Reach for **"w"** on a file you meant to keep and **the old contents are gone** - it does not warn you.

"w" write - creates it, or REPLACES it
everything already in the file is thrown away

```
with open("demo_notes.txt", "w") as f:  
    f.write("first line\n")  
    f.write("second line\n")
```

```
with open("demo_notes.txt", "r") as f:  
    content = f.read()
```

"r" read - the file must already exist
this is the default if you pass no mode

PRINTING content

```
first line  
second line
```

"a" append - adds to the end, keeps the rest
coming up in Step-02

with is why there is no `f.close()` here. It closes the file when the block ends - even if an error happens partway through.

Two habits worth forming now: always pass **encoding="utf-8"** so text reads the same on every computer, and tidy up a demo file with **`Path("demo_notes.txt").unlink()`**.

Files: Write a Text File, Then Read It Back

A text file keeps your data on disk after the program ends; `open()` saves text to it and reads it back into your program.

#	CONCEPT	SAMPLE CODE
1	Write a file <code>with open</code> Mode "w" creates or replaces; with closes it for you.	<pre>f.write("first line\n")</pre>
2	Read it back with mode <code>"r"</code> Load the whole file as one string.	<pre>content = f.read()</pre>
3	Clean up the file <code>.unlink()</code> Last step: remove the demo file; missing_ok=True if it is already gone.	<pre>Path("notes.txt").unlink()</pre>

Files: **append**, and three ways to read **lines**

A log grows; it does not get rewritten

Mode **"a"** adds to the end and keeps what was there. That one letter is the difference between a log and a file you just destroyed.

open(path, "a")

keeps every old line, adds yours at the end - what a log needs

open(path, "w")

throws the old lines away, starts empty - only when you mean it

ONE LINE AT A TIME

```
for line in f:
    print(line.strip())
```

Kalyan logged in
Kalyan opened the report
Kalyan logged out

Never loads the whole file - the right choice for a big one. `strip()` drops the newline.

EVERY LINE AS A LIST

```
f.readlines()
```

```
['alpha\n', 'beta\n', 'gamma\n']
```

Watch the `\n` on every item. This is why `line == "alpha"` quietly fails.

CLEAN LINES AS A LIST

```
f.read().splitlines()
```

```
['alpha', 'beta', 'gamma']
```

Same list, no newlines. Usually the one you actually wanted.

Tidying up: **`Path("demo_log.txt").unlink()`** removes the file, and **`unlink(missing_ok=True)`** stays quiet when it is already gone instead of raising **`FileNotFoundError`**.

Files: **Append** Lines, Then Read Line by Line

Append mode adds new lines to the end of a file without erasing the old ones, then you read them back one line at a time.

#	CONCEPT	SAMPLE CODE
1	Append mode <code>"a"</code> Adds new lines to the end and keeps the old entries.	<code>open("demo_log.txt", "a")</code>
2	Read line by line <code>for line in f</code> One line at a time, so the whole file never loads.	<code>print("line:", line.strip())</code>
3	Read every line at once <code>.readlines()</code> The whole file as a list: costs memory, buys len() and indexing.	<code>all_lines = f.readlines()</code>
4	Rewind the cursor <code>seek(0)</code> A second read returns nothing until you rewind to the start.	<code>f.seek(0)</code>

Files: **json** - the **s** means string

Four names, one letter apart - here is the rule that settles it

The **s** stands for **string**. No **s** means it works on a **file**. That is the only thing separating the four.

PYTHON DICT -> JSON

JSON -> PYTHON DICT

to a
STRING
with s

json.dumps

gives you the JSON as text, no file

```
{"name": "Kalyan", "age": 30,  
  "languages": ["Python", "Bash"]}
```

json.loads

turns that text back into a dict

```
back["languages"][0] -> Python
```

to a **FILE**
no s

json.dump

writes the dict into an open file

```
json.dump(person, f, indent=2)
```

json.load

reads the file back as a real dict

```
{'name': 'Kalyan', 'age': 30,  
  'languages': ['Python', 'Bash']}
```

Look at the quotes

JSON text uses "double quotes"; a Python dict prints with 'single' ones

type(text) is str

dumps gives you a string you can send or log, not a saved file

Use **dump / load** when the data belongs on disk - settings, a saved record. Use **dumps / loads** when it is going somewhere else, like an API request or a log line.

Files: Save and Load Data with JSON

JSON is a simple text format for structured data - dicts, lists, strings, and numbers - so you can save it to a file and read it back later.

#	CONCEPT	SAMPLE CODE
1	Dict to string and back <code>json.dumps()</code> <code>json.loads()</code> Turn a dict into a JSON string for sending or logging.	<pre>text = json.dumps(person)</pre>
2	Save a dict to a file <code>json.dump()</code> Write a Python dict to a file as readable JSON.	<pre>json.dump(person, f, indent=2)</pre>
3	Load a file into a dict <code>json.load()</code> Read the JSON file back into a Python dict.	<pre>loaded = json.load(f)</pre>

Files: one **path**, four parts

Build paths with / and never type a slash again

A **Path** knows it is a path, so joining with / makes the right separator on Windows, Mac and Linux. **Never glue path strings together by hand.**

```
from pathlib import Path
p = Path("reports") / "app_files" / "demo_note.txt"      # reports/app_files/demo_note.txt
```

reports/app_files/demo_note.txt

p.parent
reports/app_files

p.name
demo_note.txt

p.stem
demo_note

p.suffix
.txt

p.exists()

asks the operating system if it is really there - this one gives **False**, it was never created

write_text() / read_text()

a whole small file in one line, no **with** block needed

A path object is not a string - it knows its own parts. **p.name** gives you the filename without any splitting, searching or slicing.

Files: Paths with pathlib

pathlib represents a file or folder path as an object you can build, inspect, and use the same way on Windows, Mac, and Linux.

#	CONCEPT	SAMPLE CODE
1	Build a path <code>/</code> Join folder and file parts with the / operator.	<pre>p = Path("reports") / "app_files" / "demo_note.txt"</pre>
2	Parts of a path <code>.name</code> <code>.stem</code> Read each piece: <code>.name</code> , <code>.stem</code> , <code>.suffix</code> , <code>.parent</code> .	<pre>print("name: ", p.name)</pre>
3	Write and read <code>.write_text()</code> <code>.read_text()</code> Save or read a whole small file in one line.	<pre>note.write_text("saved with one line\n")</pre>
4	Does it exist? <code>.exists()</code> False before you write it, True after, False once removed.	<pre>print(p.exists())</pre>

Files: Save Next to Your Script

A bare file name lands wherever you RAN the program. `__file__` is the path of the running script, so a path built from it always points to the same place.

#	CONCEPT	SAMPLE CODE
1	Pin a path to your code <code>__file__</code> A bare name follows the terminal; this follows your script.	<code>Path(__file__).parent / "notes.txt"</code>
2	A folder beside your code <code>mkdir()</code> Make the folder, then write the file into it.	<code>folder.mkdir(parents=True, exist_ok=True)</code>